

PC Bang Manager

Cours complet & Fiche de révision

Comprendre toute l'application, du bouton jusqu'à la base de données

Electron

Angular 19

Prisma

SQLite

Préparation à l'examen oral SGBD

Sommaire

PARTIE A — La fiche : le schéma relationnel

- A.1 — Les 9 tables
- A.2 — Les relations (cardinalités)
- A.3 — Les deux tables de jonction
- A.4 — Les règles de suppression (onDelete)

PARTIE B — Le cours : comprendre tout le projet

- Chapitre 0 — Le vocabulaire
- Chapitre 1 — Les 4 technologies et pourquoi elles cohabitent
- Chapitre 2 — La grande architecture (schéma maître)
- Chapitre 3 — Angular pour les novices
- Chapitre 4 — Electron pour les novices (le pont)
- Chapitre 5 — Prisma & SGBD pour les novices
- Chapitre 6 — LE FLUX, ligne par ligne
- Chapitre 7 — Questions types d'oral (avec réponses)

PARTIE A — La fiche : le schéma relationnel (9 tables)

A.1 — Les tables, une par une

Table	Clé primaire	Colonnes	Valeurs par défaut
Abonnement	id_abonnement	type, prix?, duree?	—
Client	id_client	nom?, prenom?, email?, telephone?, id_abonnement? (FK)	—
Poste	id_poste	numero_poste?, statut, type?	statut = "Libre"
Reservation	id_reservation	id_client? (FK), id_poste? (FK), date_debut?, duree?, statut	statut = "EnAttente"
Categorie	id_categorie	nom	—
Snack	id_snack	nom, prix, stock, id_categorie? (FK)	stock = 0
Jeu	id_jeu	titre, editeur?, genre?, annee?	—
ReservationSnack	(id_reservation + id_snack)	quantite, prix_unitaire	quantite = 1
PosteJeu	(id_poste + id_jeu)	(aucune autre)	—

? = colonne optionnelle (peut être vide / **NULL**). **FK** = clé étrangère (pointe vers une autre table).

A.2 — Les relations (cardinalités)

```
Abonnement 1 —< N Client      (un abonnement pour plusieurs clients)
Client      1 —< N Reservation (un client a plusieurs réservations)
Poste       1 —< N Reservation (un poste a plusieurs réservations)
Categorie   1 —< N Snack       (une catégorie regroupe plusieurs snacks)

Reservation N >—< N Snack      via ReservationSnack (jonction RICHE)
Poste       N >—< N Jeu         via PosteJeu          (jonction PURE)
```

- **1 —< N** (un-à-plusieurs) : le côté « N » porte la clé étrangère. Ex : chaque **Reservation** contient un **id_client**.
- **N >—< N** (plusieurs-à-plusieurs) : impossible directement en SQL, on crée une **table de jonction** au milieu, avec une clé primaire **composite** (les deux FK ensemble).

A.3 — Les deux tables de jonction (point fort à l'oral SGBD)

	ReservationSnack	PosteJeu
Relie	Réservation ↔ Snack	Poste ↔ Jeu
Type	riche (porte des données)	pure (juste le lien)
Colonnes en plus	quantite, prix_unitaire	aucune
Clé primaire	(id_reservation, id_snack)	(id_poste, id_jeu)

- **Riche vs pure** : **PosteJeu** dit juste « ce jeu est installé sur ce poste ». **ReservationSnack** dit « dans cette réservation, on a commandé X snacks à tel prix » → elle porte des infos en plus.
- **prix_unitaire figé** : on stocke le prix au moment de la commande. Si le prix change demain, les anciennes commandes gardent l'ancien prix.

A.4 — Les règles de suppression (onDelete) — très demandé

Relation	Règle	Traduction
Client → Abonnement	SetNull	Supprimer un abonnement → les clients restent, <code>id_abonnement</code> devient NULL.
Reservation → Client	SetNull	Supprimer un client → ses réservations restent sans client.
Reservation → Poste	SetNull	Idem pour le poste.
Snack → Catégorie	SetNull	Supprimer une catégorie → les snacks restent, sans catégorie.
ReservationSnack → Reservation	Cascade	Supprimer une réservation → ses lignes de snacks sont supprimées avec.
ReservationSnack → Snack	Restrict	Interdit de supprimer un snack déjà commandé (protection).
PosteJeu → Poste / Jeu	Cascade	Supprimer un poste (ou un jeu) → les liens d'installation disparaissent.

Les 3 comportements à retenir : **SetNull** = garde la ligne, vide juste le lien. **Cascade** = supprime aussi ce qui dépend de moi. **Restrict** = interdit la suppression tant qu'il existe des dépendances.

PARTIE B — Le cours : comprendre tout le projet de zéro

Objectif : qu'à la fin, en lisant n'importe quel fichier du flux, tu saches dire à voix haute ce que fait chaque ligne et pourquoi elle est là.

Chapitre 0 — Le vocabulaire (à lire en premier, lentement)

Avant le code, les mots. Sans eux, le reste reste flou.

- **Frontend (« le front »)** : la partie visible — boutons, formulaires, listes. Chez toi = **Angular**.
- **Backend (« le back »)** : la partie cachée qui travaille (calculs, base de données). Chez toi = le **processus Main d'Electron + Prisma**.
- **Electron** : une techno pour faire une application de bureau avec des technos web. Electron lance **deux programmes séparés** : un pour l'écran (*render*), un pour les coulisses (*main*).
- **Angular** : une boîte à outils (framework) pour construire des interfaces. On y écrit des **composants**.
- **Prisma** : un **ORM**. Un traducteur : tu écris du JavaScript simple (`db.client.create(...)`) et il génère le **SQL** (`INSERT INTO Client...`) à ta place.
- **SQLite** : un moteur de base de données léger qui range tout dans un seul fichier (`dev.db`). Pas de serveur à installer.
- **TypeScript** : du JavaScript avec des types. « type » = on précise qu'une variable est un `number`, un `string`, un `Client` ... L'éditeur prévient si on se trompe. Fichiers en `.ts`.
- **DTO** (Data Transfer Object) : un objet juste pour transporter des données. Ex : `ClientCreateDto` = les champs pour créer un client (sans l' `id`, généré par la base).
- **IPC** (Inter-Process Communication) : le moyen pour les deux programmes d'Electron de se parler par messages.
- **Asynchrone / async / await / Promise** : une **Promise** est une « promesse de résultat plus tard ». **await** = « attends ce résultat avant de continuer » sans bloquer l'écran. Une fonction **async** peut contenir des `await`.

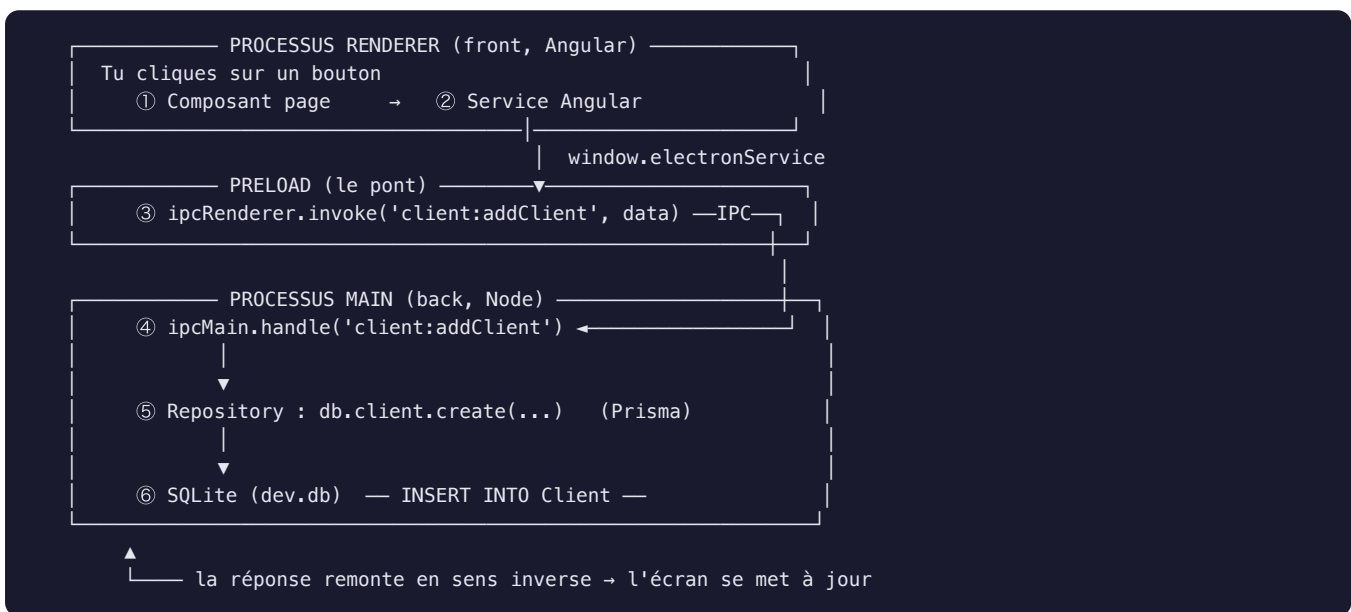
Chapitre 1 — Les 4 technologies et pourquoi elles cohabitent

Ton appli combine 4 briques. Chacune résout **un** problème :

Brique	Problème qu'elle résout	Où dans le projet
Electron	« Je veux une vraie appli de bureau, pas un site web. »	src/main, src/preload
Angular	« Je veux une interface réactive (boutons, formulaires). »	src/render
Prisma	« Je veux parler à la base sans écrire de SQL. »	src/main/repositories/.../*Repository.ts
SQLite	« Je veux stocker durablement mes données. »	src/main/repositories/prisma/dev.db

L'enchaînement logique : **Angular** dessine l'écran → **Electron** transporte la demande vers les coulisses → **Prisma** la traduit en SQL → **SQLite** la stocke. Et la réponse remonte.

Chapitre 2 — La grande architecture (le schéma maître)



Règle d'or : le front ne touche **jamais** directement à la base. Il **demande** au back via le pont. C'est une sécurité (un écran piraté ne peut pas attaquer la base) et une organisation (chacun son rôle).

Chapitre 3 — Angular pour les novices (le strict nécessaire)

3.1 — Un composant = un template + une classe

```

@Component({
  selector: 'app-client-list', // le "nom de balise" : <app-client-list>
  template: `... du HTML ...`, // ce qui s'affiche
})
export class ClientListComponent { ... } // la logique
  
```

Le `@Component({...})` au-dessus est un **décorateur** : une étiquette qui dit à Angular « cette classe est un composant ».

3.2 — Les signals (la réactivité)

```

clients = signal<Client[]>([]); // au départ, liste vide
this.clients.set(data);         // on CHANGE la valeur
const liste = this.clients();    // on LIT la valeur (avec parenthèses !)
  
```

La magie : dès que tu fais `.set(...)`, tout le HTML qui lit ce signal se redessine tout seul. C'est pour ça que `@for (client of clients(); ...)` se met à jour automatiquement quand on ajoute un client.

3.3 — Les @if et @for dans le template

- `@if (condition) { ... }` : affiche le bloc seulement si la condition est vraie.
- `@for (x of liste(); track x.id) { ... }` : répète le bloc pour chaque élément. `track x.id` aide Angular à identifier chaque élément (performance).

3.4 — Communication parent ↔ enfant (input / output)

La liste est le **parent** ; le formulaire et les cartes sont ses **enfants**.

Du parent vers l'enfant = input() (on descend une donnée) :

```
// dans l'enfant (formulaire) :
clientAModifier = input<Client | null>(null);
// dans le parent (liste) : on remplit cette entrée
<app-client-form [clientAModifier]="clientSelectionne()" />
```

De l'enfant vers le parent = output() (on remonte un événement) :

```
// dans l'enfant :
sauvegarde = output<Client>();
this.sauvegarde.emit(created); // "j'ai fini, voilà le client créé"
// dans le parent : on écoute cet événement
<app-client-form (sauvegarde)="onSauvegarde($event)" />
```

Phrase mémo : crochets `[...]` = ça descend ; parenthèses `(...)` = ça remonte. (`(click)` est juste le cas où l'événement qui remonte est un clic.)

3.5 — Service + injection de dépendances

```
constructor(private clientService: ClientService) {}
//           ↑ Angular crée et fournit le service tout seul
```

Le composant n'instancie pas le service : Angular le lui **fournit** via le constructeur (l'**injection de dépendances**). Avantage : un seul endroit centralise les appels au back.

3.6 — Le routeur (la navigation)

- `app.routes.ts` = table « URL → composant ».
- `routerLink="/clients"` dans la navbar = lien cliquable.
- `<router-outlet />` dans `app.component.ts` = le trou où la page courante s'affiche.

Chapitre 4 — Electron pour les novices (le pont)

4.1 — Trois fichiers, trois rôles

- **main.ts** (processus Main) : démarre l'appli, crée la fenêtre, parle à la base. C'est Node.js, il a tous les droits.
- **renderer** (Angular) : la page. C'est un navigateur, il n'a aucun droit sur la machine (sécurité).
- **preload.ts** : le **seul** fichier qui voit les deux côtés. Il expose des fonctions précises au front.

4.2 — contextBridge : créer la passerelle

```
contextBridge.exposeInMainWorld("electronService", { clients: clientService(), ... });
```

Cette ligne **pose** un objet `electronService` sur la page web. Désormais le front peut écrire `window.electronService.clients.addClient(...)`. C'est la seule porte ouverte vers le back — et elle ne laisse passer que ce qu'on a choisi.

4.3 — IPC : invoke (front) ↔ handle (back)

```
// PRELOAD (front) — on ENVOIE et on ATTEND la réponse :
ipcRenderer.invoke('client:addClient', client);

// MAIN (back) — on ÉCOUTE ce nom précis :
ipcMain.handle('client:addClient', async (_, client) => { ... });
```

Le nom `'client:addClient'` est l'élément le plus important du flux. Il doit être identique au caractère près des deux côtés. C'est l'adresse postale du message. Convention : `'entité:action'`.

Chapitre 5 — Prisma & SGBD pour les novices

5.1 — Un « model » = une table

```
model Client {
  id_client      Int      @id @default(autoincrement()) // clé primaire auto
  nom            String?           // ? = optionnel (NULL possible)
  id_abonnement  Int?
  abonnement     Abonnement? @relation(fields: [id_abonnement],
                                     references: [id_abonnement], onDelete: SetNull)
}
```

- `@id` = clé primaire. `@default(autoincrement())` = la base génère 1, 2, 3...
- `String?` = texte optionnel. `Int` (sans `?`) = entier obligatoire.
- `@relation(...)` = définit le lien vers une autre table + la règle `onDelete`.

5.2 — Les 5 méthodes Prisma et leur SQL

Méthode Prisma	SQL généré	Sens
<code>findMany()</code>	<code>SELECT * FROM ...</code>	lire plusieurs lignes
<code>findUnique({ where })</code>	<code>SELECT ... WHERE id = ?</code>	lire une ligne précise
<code>create({ data })</code>	<code>INSERT INTO ...</code>	ajouter
<code>update({ where, data })</code>	<code>UPDATE ... SET ... WHERE id = ?</code>	modifier
<code>delete({ where })</code>	<code>DELETE FROM ... WHERE id = ?</code>	supprimer

Et l'option **include** :

```
this.db.client.findMany({ include: { abonnement: true } });
```

include = **fais la jointure** et ramène aussi l'objet lié (équivalent d'un `JOIN` SQL). C'est ce qui permet à la carte client d'afficher le badge de l'abonnement.

5.3 — Pourquoi un ORM plutôt que du SQL ?

Sans Prisma, tu écrirais le `INSERT INTO Client (...) VALUES (...)` à la main, avec des risques d'erreurs et d'injection SQL. Prisma génère ce SQL pour toi, le sécurise, et te rend un objet JavaScript propre.

Argument d'oral : « Prisma me donne la sécurité, la lisibilité et le typage, tout en générant le SQL standard derrière. »

Chapitre 6 — LE FLUX, ligne par ligne (le cœur du cours)

On refait « Ajouter un client » en expliquant chaque étape comme si tu la lisais à l'oral.

① Le bouton — `client-list.component.ts`

```
<button class="btn btn-primary" (click)="ouvrirFormulaire()">+ Ajouter un client</button>
```

À dire Le bouton écoute le clic avec `(click)` et appelle la méthode `ouvrirFormulaire()` de la classe.

```
ouvrirFormulaire() {  
  this.clientSelectionne.set(null); // null = mode CRÉATION  
  this.afficherFormulaire.set(true); // on demande l'ouverture du formulaire  
}
```

À dire On met le signal `afficherFormulaire` à `true`. Comme le `@if` du template surveille ce signal, le formulaire apparaît automatiquement. `clientSelectionne` à `null` indique qu'on crée (et non qu'on modifie).

① Saisie et soumission — `client-form.component.ts`

```
form = this.fb.group({  
  nom: ['', Validators.required],  
  email: ['', [Validators.required, Validators.email]],  
  // ...  
});
```

À dire Formulaire réactif : chaque champ a ses validateurs. Tant qu'un champ obligatoire est vide, `form.invalid` est vrai et le bouton Sauvegarder reste désactivé.

```
async onSubmit() {  
  if (this.form.invalid) return; // sécurité  
  const values = this.form.value; // les champs saisis  
  const client = this.clientAModifier();  
  if (client) { /* update */ }  
  else {  
    const created = await this.clientService.addClient(values as ClientCreatedDto);  
    this.sauvegarde.emit(created); // on prévient le parent (ÉTAPE 7)  
  }  
}
```

À dire À la soumission, comme `clientAModifier` vaut `null`, on tombe dans le `else` = création. On appelle le service Angular et on attend (`await`) sa réponse, puis on émet l'événement `sauvegarde` pour prévenir la liste.

② Le service Angular — `client.service.ts`

```
addClient(dto: ClientCreatedDto): Promise<Client> {  
  return window.electronService.clients.addClient(dto);  
}
```

À dire Le service ne fait que transmettre. Il appelle `window.electronService`, l'objet créé par le pont preload. C'est la dernière étape côté front.

③ Le pont (envoi) — preload/clientService.ts

```
async addClient(client: ClientCreateDto): Promise<Client> {
  return await ipcRenderer.invoke('client:addClient', client);
}
```

À dire Ici on franchit la frontière entre les deux processus. `ipcRenderer.invoke` envoie un message IPC nommé `'client:addClient'` avec les données, et attend la réponse du back.

③ bis Le branchement du pont — preload/preload.ts

```
contextBridge.exposeInMainWorld("electronService", { clients: clientService(), ... });
```

À dire C'est cette ligne qui a créé `window.electronService` utilisé à l'étape 2. C'est l'unique passerelle sécurisée exposée au front.

④ L'oreille IPC — registerClientRepository.ts

```
ipcMain.handle('client:addClient', async (_, client) => {
  try {
    return JSON.parse(JSON.stringify(await repo.addClient(client)));
  } catch (error) {
    console.error('[client:addClient]', error);
    throw new Error('Impossible de créer le client');
  }
});
```

À dire Côté back, `ipcMain.handle` écoute exactement le même nom `'client:addClient'`. Le `_` est l'expéditeur (ignoré), `client` est la donnée reçue. On délègue au repository, on **séréalise** le résultat (`JSON.parse(JSON.stringify())`) pour qu'il traverse proprement l'IPC, et un `try/catch` gère les erreurs.

⑤ La requête base — clientRepository.ts

```
async addClient(dto: ClientCreateDto): Promise<Client> {
  return this.db.client.create({ data: dto, include: { abonnement: true } });
}
```

À dire Le repository parle à SQLite via Prisma. `create` correspond à un `INSERT INTO Client`. `include: { abonnement: true }` fait la jointure pour ramener aussi l'abonnement. La ligne créée, avec son nouvel `id`, est renvoyée.

⑥ Le retour — client-list.component.ts

```
async onSauvegarde(client: Client) {
  await this.chargerClients(); // on relit toute la liste depuis la base
  this.fermerFormulaire();
}
```

À dire La réponse a remonté tout le chemin. La liste, qui écoutait `sauvegarde`, recharge les clients et ferme le formulaire. `chargerClients` remet le signal à jour, donc le `@for` redessine la grille et le nouveau client apparaît. La boucle est bouclée.

Le résumé en une phrase (à réciter) : Bouton → méthode du composant → service Angular → pont preload (`ipcRenderer.invoke`) → oreille du back (`ipcMain.handle`) → repository Prisma (`create`) → SQLite. Puis la réponse remonte et le signal Angular rafraîchit l'écran.

Chapitre 7 — Questions types d'oral (avec réponses)

Q : Où est le bouton "Ajouter un client" ?

R : Dans `src/renderer/pages/client/client-list.component.ts`, dans le template, attribut `(click)="ouvrirFormulaire()"`.

Q : Comment le front communique-t-il avec la base ?

R : Jamais directement. Il passe par le pont preload (`contextBridge` + `ipcRenderer.invoke`) qui envoie un message IPC au processus Main, lequel écoute avec `ipcMain.handle` et exécute la requête Prisma.

Q : C'est quoi 'client:addClient' ?

R : Le nom du canal IPC. Il relie l'envoi (`invoke` , côté preload) à la réception (`handle` , côté main). Il doit être identique des deux côtés.

Q : Pourquoi séparer `registerClientRepository` et `clientRepository` ?

R : Séparation des responsabilités : le `register` gère la communication IPC et les erreurs ; le `repository` gère uniquement les requêtes à la base.

Q : C'est quoi un signal ?

R : Une variable réactive Angular. Quand on change sa valeur avec `.set()`, le HTML qui la lit se redessine automatiquement.

Q : Comment passe-t-on une donnée d'un composant à un autre ?

R : Parent → enfant avec `input()` et `[crochets]` ; enfant → parent avec `output()` + `.emit()` et `(parenthèses)`.

Q : C'est quoi Prisma, et un include ?

R : Prisma est l'ORM qui traduit le JavaScript en SQL. `include` réalise une jointure pour ramener aussi les objets liés.

Q : Que se passe-t-il si je supprime une catégorie qui a des snacks ?

R : Grâce à `onDelete: SetNull`, les snacks ne sont pas supprimés : leur `id_categorie` passe simplement à `NULL`.